

Sasho Abdulrahim Derama

Senior AI Product & Systems Engineer

Agent infrastructure | verified software change | local-first developer platforms | native and real-time products

Sofia, Bulgaria | GitHub: <https://github.com/gonzo-max2> | MAYA case study: <https://github.com/gonzo-max2/maya-codex-nexus> | Aegis: <https://github.com/gonzo-max2/aegis-unified-0.9.0> | NOVA (private evaluation repository)

Print-ready CV: [cv.html](#) | PDF CV: [Sasho-Abdulrahim-Derama-CV.pdf](#)

EXECUTIVE PROFILE

Senior multidisciplinary engineer and product builder working where probabilistic AI meets deterministic software authority. I turn ambiguous product intent into inspectable systems with explicit state, bounded execution, verification, rollback, recovery, and operator control.

Hands-on across Python, FastAPI, Pydantic, TypeScript, React, Node.js, C++23, JUCE, Rust/Tauri, Linux, Docker, SQLite, Git, and GitHub Actions. Strongest in agent runtimes, developer tooling, local-first architecture, native applications, real-time interfaces, and evidence-backed release engineering.

SELECTED ENGINEERING OUTCOMES

- Architected a three-layer product family: MAYA for operator experience and mission control, NOVA for proof-carrying source change and exact Git admission, and Aegis for policy-gated execution and runtime recovery.
- Built Aegis as a pre-production local agent runtime with one canonical authority model exposed through a scrollback-first CLI and a React Studio.
- Built NOVA's local verified-change path around source identity, bounded patches, durable transactions, independent verification, signed change records, and exact Git admission.
- Recorded deterministic NOVA baseline validation including 317,629 generated assertions, 95 browser-harness assertions, 99 local-agent checks, 97 hostile control-plane checks, and 1,000 compatibility operations.
- Directed MAYA's product architecture and public case study using actual sanitized interface captures, explicit maturity boundaries, and an evidence register rather than invented product claims.

INDEPENDENT PRODUCT ENGINEERING

Aegis - Principal Engineer / Product Creator | 2026-Present

Policy, provenance, and reversible execution for semantic agents

- Designed and implemented a local-first runtime between AI models and the operating system. Deterministic code owns permissions, risk classification, approvals, path confinement, process ownership, receipts, rollback data, and terminal completion.
- Unified two operator surfaces over the same canonical runtime events: a prompt-toolkit/Rich terminal interface and a React Studio for plans, changes, approvals, failures, specialist work, and proof.
- Implemented persistent missions, checkpoints, bounded continuation, cancellation, provider recovery, exact tool receipts, file deltas, proof manifests, and explicit non-success states.
- Built permission-scoped specialist routing, reduced tool registries, JIT skill activation, workspace-scoped verified episodic memory, and local/hosted model adapters.
- Hardened local execution with workspace boundaries, process-group cleanup, output limits, optional Docker isolation, mandatory SSH host verification, secret redaction, and SSRF-aware web access.
- Published the implementation under Apache-2.0 with reproducible documentation, CI, release chronology, real interface evidence, and explicit security limitations.

Current boundary: serious pre-production alpha for one operator on a trusted workstation; not presented as an independently audited multi-tenant security boundary.

NOVA - Product Architect / Full-Stack AI Engineer | 2026-Present

Proof-carrying change control for machine-generated software

- Built a React/Vite Studio and authenticated loopback-first Node agent for repository-aware source inspection, bounded change proposals, isolated checks, durable apply, recovery, and rollback.
- Implemented semantic source analysis, runtime/source identity, exact-text patch preconditions, transaction journals, independent verification, and local Ed25519 change certificates.
- Developed a reference `@nova/change-control-plane@9.0.0` package for signed policy, evidence quorum, admission receipts, and exact base/head Git pre-receive enforcement.
- Built a three-candidate Proof Gallery and orchestration core with explicit human selection, evidence visibility, local persistence, confirmed apply, local branch/commit creation, and separately confirmed remote push.
- Added a persistent Scenario Lab for supported route, viewport, theme, auth, data, loading, error, and offline conditions.

NOVA - Product Architect / Full-Stack AI Engineer | 2026-Present (CONTINUED)

- Maintained explicit non-claims around cloud tenancy, RBAC, KMS/HSM custody, installed GitHub integration, independent certification, and universal framework support.

Recorded validation boundary: self-authored deterministic repository evidence tied to stated baselines and fixtures; not independent certification or customer-scale proof.

MAYA - Independent Product Creator / Systems Designer | 2026-Present

Operator-controlled AI engineering workspace

- Directed the product around a mission lifecycle: frame, observe, plan, authorize, execute, verify, and recover.
- Designed distinct Mission and Decisions workspaces so task state, evidence, and consequential authority do not collapse into one conversational surface.
- Defined product principles for reviewable change, honest failure, recoverable history, calm operational UX, and local-first privacy.
- Built the public case study from real sanitized product captures while keeping private source, provider configuration, credentials, diagnostics, and operational history out of the public repository.
- Defined a clean responsibility split between MAYA's operator experience, NOVA's verified source-change layer, and Aegis's governed execution runtime without claiming completed integration where evidence is absent.

ADDITIONAL PRODUCT SYSTEMS

- DERAMA Studio - native C++23/JUCE DAW and audio plug-in platform focused on real-time-safe DSP, plug-in hosting, high-performance rendering, and instrument-grade interaction.
- TripCab / Vozime - React Native and Expo mobility product exploring authoritative trip lifecycle state, rider/driver workflows, offline recovery, safety actions, accessibility, and responsive mobile UX.
- Forensic recovery and QA tooling - evidence-oriented recovery workflows, repository auditing, UI diagnostics, accessibility checks, and deterministic validation harnesses.

EARLIER PROFESSIONAL EXPERIENCE**Google Account Recovery Agent**

- Handled identity and access recovery cases where privacy, procedural accuracy, abuse prevention, and account safety were central.
- Evaluated ownership evidence, distinguished routine recovery from higher-risk scenarios, and escalated when authority was insufficient.
- Documented decisions and communicated outcomes without exposing sensitive user information.

eBay Catalog Agent

- Created and maintained structured product records across categories, attributes, identifiers, and variants.
- Normalized inconsistent source data into controlled taxonomies and schema fields.
- Resolved duplicates, conflicts, missing data, and quality exceptions through repeatable checks and disciplined escalation.

TECHNICAL DEPTH

- Languages: Python, TypeScript, JavaScript, C++23, SQL, Bash; working knowledge of Rust
- AI and agent systems: tool calling, JSON Schema, model routing, Ollama, hosted-provider adapters, context compaction, approval gates, evidence and provenance, local inference
- Frontend and product: React, React Native, Expo, Vite, accessible interaction, responsive systems, WebSocket/SSE state, Playwright
- Backend and data: FastAPI, Pydantic, Node.js, REST, SQLite, durable state, event streams, transactional workflows
- Native and real-time: JUCE, CMake, Ninja, Tauri, audio/DSP architecture, GPU-oriented UI systems
- Platform and release: Linux, Docker, Git, GitHub Actions, SSH, process isolation, SBOMs, pinned CI actions, release verification
- Quality: pytest, Vitest, TypeScript, Ruff, ESLint, fuzzing, concurrency and chaos testing, accessibility and visual harnesses
- Languages: Bulgarian - native; English - professional

OPERATING STRENGTHS

First-principles architecture | technical product direction | system boundaries | failure and recovery design | skeptical QA | evidence-backed claims | high-density interface design | cross-domain implementation

PUBLIC EVIDENCE BOUNDARY

Public repositories contain implementation, architecture, validation artifacts, real sanitized interface captures, and explicit non-claims. Recorded check counts are self-authored engineering evidence tied to specific commits, fixtures, and environments. They are not represented as independent security certification, customer adoption, production-scale benchmarking, or universal correctness.